

DiscoGMPI

User Guide and Technical Reference

DiscoGMPI project

Version 0.1.0
June 2026

About this guide

This manual describes the `DiscoGMPI` software as implemented in the repository at the time of writing. It covers the mathematical model, discontinuous Galerkin (DG) discretization, Maxwell formulations, MPI decomposition, installation, mesh preparation, public workflows, examples, tests, benchmarks, diagnostics, and known limitations.

`DiscoGMPI` is the distributed-memory development line derived from `DiscoG3D`. It is a separate Julia package and repository. Work performed in `DiscoGMPI` does not overwrite the original `DiscoG3D` source tree.

Scope. The principal production path documented here is the source-free, three-dimensional Maxwell system on affine tetrahedral meshes, with explicit time integration. The MPI path supports piecewise-constant isotropic material properties, PEC, PMC, first-order absorbing, and translational periodic boundaries. It also supports the six-field nonlinear PML of Abarbanel, Gottlieb, and Hesthaven for dimensionless homogeneous vacuum problems. Impressed sources remain future work.

Notation. Superscripts $-$ and $+$ denote the local and neighboring traces at a face. The unit normal $\mathbf{n}$ points out of the minus element. The polynomial degree is denoted by N , the number of volume nodes per tetrahedron by N_p , and the number of nodes per triangular face by N_{fp} .

Contents

About this guide	i
1 Overview	1
1.1 Purpose	1
1.2 Repository layout	1
1.3 Supported execution modes	2
2 Mathematical Model	3
2.1 Source-free Maxwell equations	3
2.2 Electromagnetic energy	3
2.3 PEC boundary condition	3
2.4 PMC and absorbing boundary conditions	4
3 Tetrahedral DG Discretization	5
3.1 Mesh and approximation space	5
3.2 Affine mapping and physical derivatives	5
3.3 Topology, traces, and lifting	5
3.4 Hesthaven–Warburton formulation	6
3.5 Poisson-bracket formulation	6
4 Time Integration	8
4.1 Explicit Runge–Kutta methods	8
4.2 Explicit partitioned symplectic methods	8
4.3 CFL estimate	9
5 Distributed-Memory Design	10
5.1 Element ownership and face halos	10
5.2 Face orientation	10
5.3 Maxwell communication	10
5.4 Root-only mesh distribution	11
5.5 Replicated constructor	12
6 Installation and Environment	13
6.1 Prerequisites	13
6.2 Instantiate the Julia environment	13
6.3 Check MPI	13
6.4 Thread configuration	14
7 Mesh Preparation	15
7.1 RawVTUMesh format	15
7.2 Boundary and material tags	15
7.3 Distributed periodic boundaries	16

7.4	Inspect a VTU mesh	16
7.5	Generate a METIS mesh	16
7.6	Visualize a partition	17
7.7	Mesh-quality requirements	17
8	Programming with DiscoGMPI	18
8.1	Construct a serial DG discretization	18
8.2	Select the threaded backend	18
8.3	Create an initial Maxwell field	18
8.4	Run a serial Hesthaven–Warburton case	19
8.5	Run a Poisson-bracket case	19
8.6	Build and run an MPI case	19
8.7	Distributed Poisson-bracket run	20
8.8	Energy diagnostics	21
8.9	Global-to-local test initialization	21
9	Running Included Examples	22
9.1	Distributed Maxwell demonstration	22
9.2	Distributed periodic Poisson-bracket traveling wave	22
9.2.1	Generating meshes with a configurable x resolution	23
9.2.2	Automatic METIS partitioning	24
9.2.3	Polynomial-order requirement	24
9.2.4	Diagnostics and production output	24
9.3	Stationary PEC cube	25
9.4	Stationary central-flux ESPRK case	25
9.5	Stationary Poisson-bracket ESPRK case	25
9.6	Upwind convergence study	26
9.7	Central-flux partitioned convergence study	26
9.8	Poisson-bracket threaded convergence study	26
10	Testing	27
10.1	Package tests	27
10.2	MPI face permutation test	27
10.3	MPI METIS partition tests	27
10.4	Distributed Maxwell equivalence tests	27
10.5	Recommended pre-commit sequence	28
11	Benchmarking	29
11.1	Single benchmark	29
11.2	Thread scaling sweep	29
11.3	Plot scaling data	29
11.4	MPI performance measurements	30
12	Output and Visualization	31
12.1	Serial VTU field output	31
12.2	Partition output	31
12.3	Distributed mesh files	31
12.4	Checkpoint and restart	32
12.5	ParaView time series	32
12.6	Run and partition metadata	32

13 Diagnostics and Troubleshooting	33
13.1 MPI run hangs	33
13.2 Wrong interface flux or rank-dependent error	33
13.3 Missing or incorrect boundary conditions	33
13.4 Periodic driver fails during partitioning	34
13.5 Unstable time integration	34
13.6 Poisson-bracket errors	34
13.7 Threaded run is slower	34
13.8 Memory use is larger than expected	35
14 Current Limitations and Roadmap	36
14.1 Current limitations	36
14.2 Recommended next development stages	36
A API Quick Reference	38
B Command Quick Reference	40
C Reproducibility Checklist	42
License	43

List of Tables

4.1	Explicit Runge–Kutta schemes available through <code>explicit_rk_scheme(order)</code> .	8
-----	---	---

Chapter 1

Overview

1.1 Purpose

DiscoGMPI provides:

- nodal DG operators for affine tetrahedral meshes;
- source-free Maxwell volume and surface operators;
- Hesthaven–Warburton strong-form and Poisson-bracket formulations;
- central and upwind Maxwell fluxes where mathematically supported;
- serial and Julia-threaded local execution backends;
- explicit Runge–Kutta methods of orders one through five;
- explicit partitioned symplectic Runge–Kutta (ESPRK) schemes of orders one through six;
- METIS element partition input;
- owned-element and one-layer face-halo mesh construction;
- MPI face orientation and nodal permutation handling;
- nonblocking exchange of the six Maxwell traces;
- six-field nonlinear PML damping without auxiliary variables;
- distributed energy reductions and distributed time integration;
- partition visualization, regression tests, convergence cases, and backend benchmarks.

1.2 Repository layout

Path	Purpose
<code>src/DiscoGMPI.jl</code> <code>src/solver/</code>	Package module, includes, and exported API. Serial and threaded DG/Maxwell data structures and kernels.
<code>src/DistributedMesh3D.jl</code>	Ownership, ghost elements, local/global maps, face adjacency, and neighbor communication tables.

Path	Purpose
<code>src/DistributedDG.jl</code>	Rank-local DG construction, root-only mesh distribution, Maxwell trace exchange, distributed right-hand side, energy, and time integration.
<code>src/NonlinearPML.jl</code>	Nonlinear PML profiles, local source terms, and serial/distributed explicit RK wrappers.
<code>src/TraceMaps.jl</code>	General MPI trace maps and face-node permutation support.
<code>src/MetisIO.jl</code>	Legacy VTK tetrahedral reader and METIS mesh writer.
<code>src/PartitionVTK.jl</code>	Legacy VTK partition visualization writers.
<code>examples/solver/</code>	Serial and threaded Maxwell examples and convergence drivers inherited from DiscoG3D.
<code>examples/distributed_maxwell.jl</code>	End-to-end MPI Maxwell demonstration.
<code>examples/meshes/</code>	Small sample mesh and METIS partition files.
<code>benchmark/solver/</code>	Serial-versus-threaded right-hand-side benchmarks and plotting scripts.
<code>test/</code>	Package tests and explicitly launched MPI tests.

1.3 Supported execution modes

Serial

One Julia process using `SerialBackend()`.

Threaded

One Julia process using `ThreadedBackend()` and a Julia thread count selected with `-threads=N` or `JULIA_NUM_THREADS=N`.

Distributed memory

Multiple Julia processes launched with `mpiexec`. Every rank owns a subset of tetrahedra and stores a one-face halo.

Hybrid MPI plus threads

The distributed constructor accepts a backend, so a threaded local backend can be selected per rank. This path should be validated on the target machine before production use; the primary MPI regression tests use the serial local backend.

Chapter 2

Mathematical Model

2.1 Source-free Maxwell equations

For scalar, positive, elementwise-constant permittivity ε_K and permeability μ_K , `DiscoGMPI` advances on each tetrahedron K

$$\varepsilon_K \frac{\partial \mathbf{E}}{\partial t} = \nabla \times \mathbf{H}, \quad (2.1)$$

$$\mu_K \frac{\partial \mathbf{H}}{\partial t} = -\nabla \times \mathbf{E}. \quad (2.2)$$

The element wave speed, impedance, and admittance are

$$c_K = \frac{1}{\sqrt{\varepsilon_K \mu_K}}, \quad Z_K = \sqrt{\frac{\mu_K}{\varepsilon_K}}, \quad Y_K = \frac{1}{Z_K}. \quad (2.3)$$

The divergence constraints

$$\nabla \cdot (\varepsilon \mathbf{E}) = 0, \quad \nabla \cdot (\mu \mathbf{H}) = 0 \quad (2.4)$$

are not solved as separate equations. Their preservation depends on the initial condition, mesh, spatial discretization, flux, and time integrator.

2.2 Electromagnetic energy

The continuous energy is

$$\mathcal{E}(t) = \frac{1}{2} \int_{\Omega} \left(\varepsilon |\mathbf{E}|^2 + \mu |\mathbf{H}|^2 \right) d\mathbf{x}. \quad (2.5)$$

The code evaluates its DG approximation element by element using the reference mass matrix and affine Jacobian determinant:

$$\mathcal{E}_h = \frac{1}{2} \sum_K |J_K| \left[\varepsilon \sum_{\alpha \in \{x,y,z\}} \mathbf{E}_{\alpha,K}^T M \mathbf{E}_{\alpha,K} + \mu \sum_{\alpha \in \{x,y,z\}} \mathbf{H}_{\alpha,K}^T M \mathbf{H}_{\alpha,K} \right]. \quad (2.6)$$

In MPI runs only owned elements contribute locally, followed by an `MPI.Allreduce` sum. Ghost elements are therefore not double counted.

2.3 PEC boundary condition

The perfect-electric-conductor condition is

$$\mathbf{n} \times \mathbf{E} = \mathbf{0} \quad \text{on } \partial\Omega_{\text{PEC}}. \quad (2.7)$$

DiscoGMPI imposes it through a reflected exterior trace:

$$\mathbf{E}^+ = -\mathbf{E}^- + 2(\mathbf{n} \cdot \mathbf{E}^-)\mathbf{n}, \quad (2.8)$$

$$\mathbf{H}^+ = \mathbf{H}^-. \quad (2.9)$$

This reverses the tangential electric field while retaining its normal component. Boundary-condition selection is tag based. A typical registry is:

```
1 registry = MaxwellBoundaryRegistry(
2     Dict{10 => DiscoGMPI.MaxwellBC_PEC},
3 )
```

An unregistered boundary tag maps to `MaxwellBC_None`; it does not silently become PEC.

2.4 PMC and absorbing boundary conditions

The perfect-magnetic-conductor state reflects the tangential magnetic field and retains the electric trace. It is selected with `MaxwellBC_PMC`. The `MaxwellBC_Absorbing` state is a first-order, impedance-matched characteristic condition: the incoming tangential characteristic is set to zero using the adjacent element impedance. It is not a PML and is most accurate for locally normally incident waves.

```
1 registry = MaxwellBoundaryRegistry(Dict(
2     10 => MaxwellBC_PEC,
3     20 => MaxwellBC_PMC,
4     30 => MaxwellBC_Absorbing,
5 ))
```

2.5 Six-field nonlinear PML

DiscoGMPI implements the primary nonlinear PML proposed by Abarbanel, Gottlieb, and Hesthaven for time-dependent electromagnetics in three dimensions. The implementation modifies the six physical Maxwell fields directly and introduces no auxiliary unknowns. In its current dimensionless homogeneous vacuum form,

$$\frac{\partial \mathbf{E}}{\partial t} = \nabla \times \mathbf{H} + (\boldsymbol{\sigma} \odot \mathbf{P}_H) \times \mathbf{H}, \quad (2.10)$$

$$\frac{\partial \mathbf{H}}{\partial t} = -\nabla \times \mathbf{E} + (\boldsymbol{\sigma} \odot \mathbf{P}_E) \times \mathbf{E}, \quad (2.11)$$

where $\odot$ denotes componentwise multiplication,

$$\mathbf{P}_H = \frac{\mathbf{E} \times \mathbf{H}}{a|\mathbf{H}|^2 + (1-a)|\mathbf{E}|^2 + \delta}, \quad (2.12)$$

$$\mathbf{P}_E = -\frac{\mathbf{E} \times \mathbf{H}}{a|\mathbf{E}|^2 + (1-a)|\mathbf{H}|^2 + \delta}. \quad (2.13)$$

The recommended and default parameter is $a = 1/2$. The positive regularization δ , whose default is 10^{-12} , prevents division by zero where both fields vanish. Users should verify that the selected value is small relative to the squared field scale of their problem.

For nonnegative damping coefficients $\boldsymbol{\sigma} = (\sigma_x, \sigma_y, \sigma_z)$, the regularized PML contribution to the pointwise energy balance is

$$\frac{1}{2} \frac{\partial}{\partial t} (|\mathbf{E}|^2 + |\mathbf{H}|^2) = -\nabla \cdot (\mathbf{E} \times \mathbf{H}) - \sum_{i=1}^3 \sigma_i (\mathbf{E} \times \mathbf{H})_i^2 \left(\frac{1}{D_H} + \frac{1}{D_E} \right), \quad (2.14)$$

with

$$D_H = a|\mathbf{H}|^2 + (1-a)|\mathbf{E}|^2 + \delta, \quad D_E = a|\mathbf{E}|^2 + (1-a)|\mathbf{H}|^2 + \delta.$$

The local PML term is therefore nonpositive.

Sign convention. The magnetic source in ?? follows Appendix A and the energy identity in the 2006 paper. The sign printed in its vector equation (2.10) is inconsistent with both. This choice is covered by a component-level regression test and a discrete energy-decay test.

Reference. S. Abarbanel, D. Gottlieb, and J. S. Hesthaven, “Non-Linear PML Equations for Time Dependent Electromagnetics in Three Dimensions,” *Journal of Scientific Computing*, vol. 28, nos. 2–3, 2006, DOI: [10.1007/s10915-006-9072-1](https://doi.org/10.1007/s10915-006-9072-1).

Damping profiles. `polynomial_pml_sigma` constructs a one-sided profile that is zero at the interior interface and rises to `sigma_max` at the outer boundary:

$$\sigma(\xi) = \sigma_{\max} \left[\text{clamp} \left(\frac{\xi - \xi_{\text{interface}}}{\xi_{\text{outer}} - \xi_{\text{interface}}}, 0, 1 \right) \right]^m.$$

Reversing the interface and outer-boundary coordinates produces a layer on the opposite side. Independent functions may be supplied for σ_x , σ_y , and σ_z .

```

1  pml = build_maxwell_nonlinear_pml(
2      distributed_dg;
3      sigma_x = (x, y, z) -> polynomial_pml_sigma(
4          x, 1.5, 2.0; sigma_max=12.0, degree=2
5      ),
6      a = 0.5,
7      regularization = 1e-12,
8  )
9
10 scheme = explicit_rk_scheme(4)
11 work = MaxwellRKWorkspace(U, scheme)
12 distributed_maxwell_nonlinear_pml_rk_step!(
13     U, work, scheme, dt, distributed_dg, registry,
14     HesthavenWarburtonFormulation(:upwind), pml,
15 )

```

The periodic wrapper is `distributed_periodic_maxwell_nonlinear_pml_rk_step!`. On MPI ranks, PML sources are added only to owned elements and ghost residuals remain zero. Because no additional state variables or traces are introduced, the existing six-component halo exchange is sufficient.

The nonlinear PML is dissipative and is therefore advanced with the explicit RK family. It must not be passed through the conservative ESPRK Poisson-bracket driver. The current PML wrapper fixes $\varepsilon = \mu = 1$; coupling this formulation consistently to heterogeneous materials is future work.

Chapter 3

Tetrahedral DG Discretization

3.1 Mesh and approximation space

The physical domain is partitioned into affine tetrahedra K . On each element, every scalar field component is represented in

$$\mathbb{P}^N(K) = \{p(x, y, z) : \deg p \leq N\}, \quad (3.1)$$

with

$$N_p = \frac{(N+1)(N+2)(N+3)}{6} \quad (3.2)$$

volume nodes. A triangular face contains

$$N_{fp} = \frac{(N+1)(N+2)}{2} \quad (3.3)$$

nodes.

The reference tetrahedron has vertices

$$(-1, -1, -1), \quad (1, -1, -1), \quad (-1, 1, -1), \quad (-1, -1, 1).$$

The implementation builds equispaced nodal points, an orthonormal polynomial basis, a Vandermonde matrix, nodal derivative matrices, an exact polynomial mass matrix, and face mass/lifting operators.

3.2 Affine mapping and physical derivatives

For an affine map $\mathbf{x} = \mathbf{x}_K(r, s, t)$, let J_K denote its Jacobian. Reference gradients are transformed by

$$\nabla_{\mathbf{x}} u = J_K^{-T} \nabla_{(r,s,t)} u. \quad (3.4)$$

The physical derivative matrices D_x, D_y, D_z and weak derivative matrices

$$S_x = MD_x, \quad S_y = MD_y, \quad S_z = MD_z \quad (3.5)$$

are precomputed for each tetrahedron. The Poisson-bracket formulation also uses S_x^T, S_y^T, S_z^T .

3.3 Topology, traces, and lifting

Faces are identified by sorted triples of global vertex IDs. A face appearing once is a boundary face; a face appearing twice is an interior face. More than two adjacent tetrahedra is treated as a non-manifold error.

For every interior face, the DG trace map stores:

- minus and plus element IDs;
- local face IDs;
- face-node indices;
- the plus-to-minus nodal permutation;
- the outward minus-side normal and face area.

Surface residuals are projected through the face mass matrix and lifted into the element volume. Conceptually, a face correction $\mathbf{f}$ contributes

$$M_K^{-1} M_{f,K} \mathbf{f} \quad (3.6)$$

to the nodal right-hand side, with the physical face area and element Jacobian included by the implementation.

3.4 Hesthaven–Warburton formulation

The strong-form volume terms directly approximate

$$\dot{\mathbf{E}}_K = \frac{1}{\varepsilon} (\nabla_h \times \mathbf{H})_K + \text{surface correction}, \quad (3.7)$$

$$\dot{\mathbf{H}}_K = -\frac{1}{\mu} (\nabla_h \times \mathbf{E})_K + \text{surface correction}. \quad (3.8)$$

Let $\llbracket \mathbf{E} \rrbracket = \mathbf{E}^+ - \mathbf{E}^-$ and $\llbracket \mathbf{H} \rrbracket = \mathbf{H}^+ - \mathbf{H}^-$. The central correction used by the code is

$$\mathbf{f}_E^c = \frac{1}{2} \mathbf{n} \times \llbracket \mathbf{H} \rrbracket, \quad (3.9)$$

$$\mathbf{f}_H^c = -\frac{1}{2} \mathbf{n} \times \llbracket \mathbf{E} \rrbracket. \quad (3.10)$$

The upwind flux adds tangential penalties:

$$\mathbf{f}_E^{up} = \mathbf{f}_E^c - \frac{Y}{2} \mathbf{n} \times (\mathbf{n} \times \llbracket \mathbf{E} \rrbracket), \quad (3.11)$$

$$\mathbf{f}_H^{up} = \mathbf{f}_H^c - \frac{Z}{2} \mathbf{n} \times (\mathbf{n} \times \llbracket \mathbf{H} \rrbracket). \quad (3.12)$$

Select the formulation with:

```
1 central = HesthavenWarburtonFormulation(MaxwellFlux_Central)
2 upwind  = HesthavenWarburtonFormulation(MaxwellFlux_Upwind)
```

The central flux is nondissipative in the ideal compatible setting. The upwind flux introduces numerical dissipation and is normally preferred when robust damping of unresolved jumps is more important than exact semidiscrete energy conservation.

3.5 Poisson-bracket formulation

The Poisson-bracket implementation is a partitioned weak-operator discretization. The electric update uses weak derivative matrices S_x, S_y, S_z , while the magnetic update uses their transposes. This pairing constructs a discrete skew structure and supports energy-compatible partitioned time stepping.

For an interior face, the implemented corrections can be summarized as

$$\mathbf{f}_E^{\text{PB}} = -\frac{1}{2\varepsilon} \mathbf{n} \times (\mathbf{H}^- - \mathbf{H}^+), \quad (3.13)$$

$$\mathbf{f}_H^{\text{PB}} = -\frac{1}{2\mu} \mathbf{n} \times (\mathbf{E}^- + \mathbf{E}^+). \quad (3.14)$$

Only the central flux is supported:

```
1 formulation = PoissonBracketFormulation()
```

Attempting to combine the Poisson-bracket formulation with the upwind flux raises an error because the upwind penalty is dissipative and does not define the implemented partitioned Poisson operator.

Chapter 4

Time Integration

4.1 Explicit Runge–Kutta methods

For a semidiscrete system $\dot{U} = R(U)$, an explicit Runge–Kutta method uses

$$U^{(i)} = U^n + \Delta t \sum_{j < i} a_{ij} K_j, \quad (4.1)$$

$$K_i = R(U^{(i)}), \quad (4.2)$$

$$U^{n+1} = U^n + \Delta t \sum_i b_i K_i. \quad (4.3)$$

Table 4.1: Explicit Runge–Kutta schemes available through `explicit_rk_scheme(order)`.

Order	Scheme	Stages
1	Forward Euler	1
2	Explicit midpoint	2
3	Classical third order	3
4	Classical fourth order	4
5	Dormand–Prince fifth-order update	7

The Dormand–Prince implementation uses the fifth-order update only. Adaptive step-size control and the embedded fourth-order estimator are not exposed.

4.2 Explicit partitioned symplectic methods

The Maxwell state is divided into electric and magnetic partitions. At every ESPRK stage, one partition is updated using its weight, the right-hand side is recomputed, and the complementary partition is updated using its own weight. Orders one through six are available:

```
1 scheme = explicit_partitioned_symplectic_rk_scheme(
2     4;
3     first_partition = :H,
4 )
```

For `PoissonBracketFormulation`, the magnetic partition must be advanced first:

```
1 formulation = PoissonBracketFormulation()
2
3 run_maxwell_partitioned_symplectic_time_steps!(
4     U,
```

```

5     dg,
6     registry,
7     formulation;
8     psrk_order = 4,
9     first_partition = :H,
10    dt = dt,
11    nsteps = nsteps,
12 )

```

Using the conventional explicit Runge–Kutta driver with the Poisson-bracket formulation is rejected by the high-level API.

4.3 CFL estimate

The code defines the element characteristic length

$$h_K = \frac{3V_K}{A_K}, \quad (4.4)$$

where V_K is the tetrahedron volume and A_K its total surface area. The recommended estimate is

$$\Delta t = \text{CFL} \frac{\min_K h_K}{(2N+1)c}. \quad (4.5)$$

It is obtained with:

```

1 dt, sizes = estimate_maxwell_dt(
2     mesh,
3     dg.geometry,
4     dg.ref;
5     CFL = 0.10,
6 )

```

This expression is an estimate, not a proof of stability for every mesh, formulation, flux, and RK scheme. Production studies should include a step-size sensitivity check.

Chapter 5

Distributed-Memory Design

5.1 Element ownership and face halos

An element partition vector assigns every global tetrahedron to a zero-based MPI rank:

$$p_e \in \{0, \dots, P - 1\}. \quad (5.1)$$

For rank r , DiscoGMPI stores:

- every element for which $p_e = r$ as *owned*;
- every off-rank tetrahedron sharing a complete face with an owned element as a *ghost*;
- the nodes required by those owned and ghost elements;
- global-to-local and local-to-global maps;
- ownership, material IDs, face neighbors, volumes, and barycenters;
- one communication record per neighboring rank.

Only a one-face halo is needed because the Maxwell DG surface operator couples elements only through face traces.

5.2 Face orientation

A triangular face admits six vertex permutations. For each inter-rank face, the distributed mesh records the orientation code that maps the neighbor ordering to the local ordering. The trace layer converts face nodes to barycentric coordinates and precomputes one nodal permutation for each of the six orientations. Received plus traces are therefore presented in the same physical point ordering as the local minus trace.

This permutation step is essential for polynomial order $N > 1$: matching only the three vertices is insufficient because edge and interior face nodes must also be reordered.

5.3 Maxwell communication

At each right-hand-side evaluation:

1. The six owned face traces $(E_x, E_y, E_z, H_x, H_y, H_z)$ are packed for every neighbor.
2. All nonblocking receives are posted.

3. All nonblocking sends are posted.
4. `MPI.Waitall` completes the exchange.
5. Received values are unpacked into the corresponding ghost face nodes.
6. The local DG right-hand side is evaluated.
7. Every ghost-element right-hand-side column is set to zero, ensuring that time integration updates owned elements only.

For a neighbor with F_{rn} interface faces, the message contains

$$6N_{fp}F_{rn} \quad (5.2)$$

double-precision values per right-hand-side evaluation.

Current computation model. The local DG kernel currently evaluates owned and stored ghost elements before zeroing ghost residuals. Communication volume is already trace-only, but a future optimization can avoid unnecessary ghost volume work and overlap communication with owned volume calculations.

5.4 Root-only mesh distribution

The scalable constructor keeps the global mesh and partition vector only on a designated root rank. Root builds each rank's owned-plus-halo package and sends that serialized package to the destination. Non-root ranks never construct or retain the complete global mesh.

```

1  using MPI
2  using DiscoGMPI
3
4  MPI.Init()
5  comm = MPI.COMM_WORLD
6  rank = MPI.Comm_rank(comm)
7  nranks = MPI.Comm_size(comm)
8
9  root_mesh = rank == 0 ? read_vtu_mesh("mesh.vtu") : nothing
10 root_partition = if rank == 0
11     readmetis_epart("mesh.mesh.epart.$nranks")
12 else
13     nothing
14 end
15
16 ddg = build_distributed_dg_from_root(
17     root_mesh,
18     root_partition,
19     2;
20     comm = comm,
21     root = 0,
22 )
23
24 MPI.Barrier(comm)
25 MPI.Finalize()

```

The file-based overload performs both reads only on root:

```
1 ddg = build_distributed_dg_from_root(  
2     "mesh.vtu",  
3     "mesh.mesh.epart.4",  
4     2;  
5     comm = MPI.COMM_WORLD,  
6 )
```

All ranks must call the constructor collectively. Root validation errors are broadcast so that malformed input fails collectively rather than leaving worker ranks blocked.

5.5 Replicated constructor

When a global `RawVTUMesh` and partition vector already exist on every rank, the compatibility constructor remains available:

```
1 ddg = build_distributed_dg(  
2     global_mesh,  
3     elem_to_rank,  
4     order;  
5     comm = MPI.COMM_WORLD,  
6 )
```

The root-only constructor is preferable for memory scaling on non-root ranks. Root still holds the complete input mesh during distribution.

Chapter 6

Installation and Environment

6.1 Prerequisites

- Julia 1.9 or later.
- An MPI implementation compatible with the MPI library selected by MPI.jl.
- An `mpiexec` command for multi-process runs.
- METIS and its `mpmetis` executable when generating new partitions.
- ParaView or another VTK viewer for visualization.
- Optional: Python 3 and Matplotlib for benchmark plots.
- Optional: a LaTeX installation to build this manual.

6.2 Instantiate the Julia environment

Run all commands from the repository root:

```
1 julia --project=. -e 'using Pkg; Pkg.instantiate()'  
2 julia --project=. -e 'using DiscoGMPI; println("DiscoGMPI loaded")'
```

The package dependencies are recorded in `Project.toml` and `Manifest.toml`. Important dependencies include `MPI.jl`, `ReadVTK.jl`, `VTKBase.jl`, `WriteVTK.jl`, `BenchmarkTools.jl`, and Julia's `LinearAlgebra` standard library.

6.3 Check MPI

```
1 mpiexec -n 2 julia --project=. -e '  
2 using MPI  
3 MPI.Init()  
4 println("rank ", MPI.Comm_rank(MPI.COMM_WORLD),  
5         " of ", MPI.Comm_size(MPI.COMM_WORLD))  
6 MPI.Barrier(MPI.COMM_WORLD)  
7 MPI.Finalize()  
8 '
```

If this command fails before `DiscoGMPI` is loaded, fix the `MPI.jl`/runtime configuration first. In particular, do not mix an arbitrary system `mpiexec` with a different MPI library selected by `MPI.jl`.

6.4 Thread configuration

Thread counts are selected when Julia starts:

```
1 julia --project=. --threads=8 my_case.jl
2 JULIA_NUM_THREADS=8 julia --project=. my_case.jl
```

Inside Julia, verify the setting with:

```
1 println(Threads.nthreads())
```

Chapter 7

Mesh Preparation

7.1 RawVTUMesh format

The primary solver mesh container is:

```
1 RawVTUMesh(  
2     points,          # 3 x number_of_points, Float64  
3     tets,            # 4 x number_of_tetrahedra, Int  
4     tris,            # 3 x number_of_boundary_triangles, Int  
5     tet_cell_ids,  
6     tri_cell_ids,  
7     cell_data,  
8 )
```

Connectivity uses Julia's one-based indexing. The XML VTU reader accepts tetrahedral volume cells and triangular surface cells and ignores unsupported cell types.

7.2 Boundary and material tags

By default, `DGDiscretization` looks for cell data named `boundary_id`. The array is indexed by the original VTK cell IDs; `tri_data(mesh, "boundary_id")` extracts its surface-triangle values.

The distributed constructors also accept:

```
1 boundary_tag_name = "boundary_id"  
2 material_tag_name = "region_id"
```

Material IDs are preserved in the distributed topology and can be converted to element coefficients:

```
1 materials = maxwell_element_materials(  
2     distributed_dg,  
3     Dict(  
4         1 => MaxwellMaterial(1.0, 1.0),  
5         2 => MaxwellMaterial(4.0, 2.0),  
6     ),  
7 )
```

The material-aware Maxwell overloads accept `materials` as a positional argument. Central fluxes use the coefficient of the updated element. The Hesthaven–Warburton upwind interface solver uses side-dependent impedance and admittance. Coefficients are currently isotropic and constant inside each tetrahedron.

7.3 Distributed periodic boundaries

`build_distributed_periodic_maxwell_exchange` gathers only owned periodic boundary descriptors, pairs shifted centroids and DG face nodes, and constructs directed rank-local faces. Remote periodic traces use a separate nonblocking MPI exchange; same-rank partners are read directly.

```
1 periodic = build_distributed_periodic_maxwell_exchange(
2     distributed_dg,
3     default_unit_box_periodic_specs();
4     materials = materials,
5 )
```

Periodic boundary tags must map to `MaxwellBC_None` in the ordinary boundary registry. When material-aware periodic fluxes are used, pass `materials` while constructing the exchange so remote partner coefficients are recorded.

7.4 Inspect a VTU mesh

```
1 julia --project=. examples/solver/read_vtu_mesh.jl path/to/mesh.vtu
```

The inspection example prints mesh dimensions, connectivity, tags, topology, geometry, mappings, basis diagnostics, face operators, trace checks, flux-face checks, and several local operator tests. It is intentionally verbose and is a useful first check for a new mesh.

7.5 Generate a METIS mesh

The sample workflow begins with a legacy ASCII VTK tetrahedral mesh:

```
1 julia --project=. examples/write_metis.jl
```

This calls:

```
1 write_metis_mesh_from_vtk(
2     "examples/meshes/tet_mesh.vtk",
3     "examples/meshes/tet_mesh.mesh",
4 )
```

The generated METIS mesh contains one tetrahedron per line. Create P partitions with:

```
1 mpmmetis examples/meshes/tet_mesh.mesh 4
```

METIS normally writes:

- `tet_mesh.mesh.epart.4`: one element partition ID per line;
- `tet_mesh.mesh.npart.4`: one node partition ID per line.

DiscoGMPI reads the element partition file. Partition IDs are expected to be zero based. For one-based files use:

```
1 parts = read_metis_epart(path; one_based_parts = true)
```

7.6 Visualize a partition

Edit NPROCS in `examples/write_partition_vtk.jl` if required, then run:

```
1 julia --project=. examples/write_partition_vtk.jl
```

The script writes:

- one global legacy VTK file with `partition_id` cell data;
- one owned-element VTK file per partition.

The principal visualization file is

`examples/visualization/partitioned_mesh.vtk.`

Open it in ParaView, color by `partition_id`, and select “Surface With Edges”.

7.7 Mesh-quality requirements

- Tetrahedra must have four valid, distinct nodes.
- Every interior face must be shared by exactly two tetrahedra.
- Boundary triangles must match tetrahedral exterior faces.
- Volumes and face areas must be positive.
- The partition vector length must equal the tetrahedron count.
- Every partition ID must lie in `0:nranks-1`.
- Every MPI rank should normally own at least one element.

Chapter 8

Programming with DiscoGMPI

8.1 Construct a serial DG discretization

```
1 using DiscoGMPI
2
3 mesh = read_vtu_mesh("mesh.vtu")
4 dg = DGDDiscretization(
5     mesh,
6     3;
7     boundary_tag_name = "boundary_id",
8     trace_tol = 1e-10,
9     backend = SerialBackend(),
10 )
```

The resulting `DGDDiscretization` contains the raw mesh, topology, geometry, affine mappings, reference element, face operators, physical operators, trace maps, flux faces, and backend.

8.2 Select the threaded backend

```
1 dg = DGDDiscretization(
2     mesh,
3     3;
4     backend = ThreadedBackend(),
5 )
```

The threaded kernels use face coloring so concurrently processed faces do not write to the same element residual.

8.3 Create an initial Maxwell field

```
1 Efun = (x, y, z) -> (
2     sin(pi * x),
3     0.0,
4     sin(pi * z),
5 )
6
7 Hfun = (x, y, z) -> (
8     0.0,
9     cos(pi * y),
10    0.0,
```

```

11 )
12
13 U = interpolate_maxwell_field(mesh, dg.ref, Efun, Hfun)

```

Each component is stored as an $N_p \times N_e$ matrix. Columns are elements and rows are element-local DG nodes.

8.4 Run a serial Hesthaven–Warburton case

```

1 registry = MaxwellBoundaryRegistry(
2     Dict{10 => DiscoGMPI.MaxwellBC_PEC},
3 )
4 formulation = HesthavenWarburtonFormulation(MaxwellFlux_Upwind)
5
6 dt, _ = estimate_maxwell_dt(
7     mesh,
8     dg.geometry,
9     dg.ref;
10    CFL = 0.10,
11 )
12
13 run_maxwell_time_steps!(
14     U,
15     dg,
16     registry,
17     formulation;
18     rk_order = 4,
19     dt = dt,
20     nsteps = 100,
21     energy_every = 10,
22 )

```

8.5 Run a Poisson-bracket case

```

1 formulation = PoissonBracketFormulation()
2
3 run_maxwell_partitioned_symplectic_time_steps!(
4     U,
5     dg,
6     registry,
7     formulation;
8     psrk_order = 4,
9     first_partition = :H,
10    dt = dt,
11    nsteps = 100,
12    energy_every = 10,
13 )

```

8.6 Build and run an MPI case

```

1 using MPI

```

```

2  using DiscoGMPI
3
4  MPI.Init()
5  comm = MPI.COMM_WORLD
6  rank = MPI.Comm_rank(comm)
7  nranks = MPI.Comm_size(comm)
8
9  root_mesh = rank == 0 ? read_vtu_mesh("mesh.vtu") : nothing
10 root_parts = rank == 0 ?
11     read_metis_epart("mesh.mesh.epart.$nranks") :
12     nothing
13
14 ddg = build_distributed_dg_from_root(
15     root_mesh,
16     root_parts,
17     2;
18     comm = comm,
19 )
20
21 Efun = (x, y, z) -> (sin(pi*x), 0.0, sin(pi*z))
22 Hfun = (x, y, z) -> (0.0, cos(pi*y), 0.0)
23 U = interpolate_maxwell_field(ddg, Efun, Hfun)
24
25 registry = MaxwellBoundaryRegistry(
26     Dict{10 => DiscoGMPI.MaxwellBC_PEC},
27 )
28 formulation = HesthavenWarburtonFormulation(MaxwellFlux_Upwind)
29
30 run_distributed_maxwell_time_steps!(
31     U,
32     ddg,
33     registry,
34     formulation;
35     rk_order = 3,
36     dt = 1e-4,
37     nsteps = 100,
38     energy_every = 10,
39 )
40
41 MPI.Barrier(comm)
42 MPI.Finalize()

```

Launch it with:

```
1 mpiexec -n 4 julia --project=. my_distributed_case.jl
```

8.7 Distributed Poisson-bracket run

```

1  run_distributed_maxwell_partitioned_symplectic_time_steps!(
2      U,
3      ddg,
4      registry,
5      PoissonBracketFormulation();
6      psrk_order = 4,
7      first_partition = :H,
8      dt = 1e-4,

```

```
9     nsteps = 100,  
10     energy_every = 10,  
11 )
```

8.8 Energy diagnostics

```
1  serial_energy = maxwell_energy(  
2      U,  
3      dg.ref,  
4      dg.mappings,  
5  )  
6  
7  global_energy = distributed_maxwell_energy(  
8      U,  
9      ddg,  
10 )  
11  
12 println(global_energy.electric)  
13 println(global_energy.magnetic)  
14 println(global_energy.total)
```

Distributed energy is collective: every rank must call it in the same order.

8.9 Global-to-local test initialization

`localize_maxwell_field` extracts owned and halo columns from a global field:

```
1 local_U = localize_maxwell_field(global_U, ddg)
```

This function is primarily useful for verification against a serial reference. For scalable production initialization, use `interpolate_maxwell_field(ddg, Efun, Hfun)` so no global field is required on worker ranks.

Chapter 9

Running Included Examples

9.1 Distributed Maxwell demonstration

```
1 mpiexec -n 2 julia --project=. examples/distributed_maxwell.jl
```

The script:

- builds a PEC cube only on rank zero;
- partitions its tetrahedra round-robin;
- distributes owned elements and face halos;
- initializes a cavity field locally;
- uses the Hesthaven–Warburton upwind formulation;
- advances two RK3 steps;
- reports ownership, neighbors, and global energy.

9.2 Distributed periodic Poisson-bracket traveling wave

The production periodic driver is:

```
1 examples/distributed_periodic_poisson_bracket_maxwell.jl
```

It uses the legacy VTK mesh `examples/meshes/periodic_box_2x1x1_nx4.vtk`, whose domain is

$$\Omega = [0, 2] \times [0, 1] \times [0, 1].$$

The mesh contains four structured hexahedral intervals in the x direction, each divided into six conforming tetrahedra, for a total of 24 tetrahedra. The periodic physical-surface tags are

Direction	Minus tag	Plus tag
x	1	2
y	3	4
z	5	6

The corresponding plus-to-minus translations are $(-2, 0, 0)$, $(0, -1, 0)$, and $(0, 0, -1)$. These tags must map to `MaxwellBC_None` in the ordinary boundary registry because the periodic surface operator handles them separately.

9.2.1 Generating meshes with a configurable x resolution

The generator `examples/meshes/generate_periodic_box_2x1x1_nx4.jl` accepts the number of structured cells in the x direction through `-nx`. Each cell is decomposed into six tetrahedra. For example:

```
1 julia --project=. \
2   examples/meshes/generate_periodic_box_2x1x1.jl \
3   --nx=8
```

This creates `examples/meshes/periodic_box_2x1x1_nx8.vtk`, with 36 points and 48 tetrahedra. The boundary tags remain 1/2, 3/4, and 5/6 for the x , y , and z periodic pairs. The general mesh sizes are

$$N_{\text{points}} = 4(n_x + 1), \quad N_{\text{tet}} = 6n_x.$$

An explicit output path may be selected with `-output=PATH`. If `-nx` is omitted, the default value is four and the generated VTK file is identical to the supplied `nx4` mesh.

The analytical solution is

$$\mathbf{E}(\mathbf{x}, t) = \left(0, 0, -\frac{kH_0}{\omega\varepsilon_0}\right) \sin(kx - \omega t), \quad (9.1)$$

$$\mathbf{H}(\mathbf{x}, t) = (0, H_0, 0) \sin(kx - \omega t), \quad (9.2)$$

with

$$k = \omega = 2\pi, \quad H_0 = 1, \quad \varepsilon_0 = \mu_0 = 1.$$

Thus the implemented fields are

$$\mathbf{E} = (0, 0, -\sin(2\pi x - 2\pi t)), \quad \mathbf{H} = (0, \sin(2\pi x - 2\pi t), 0).$$

Run two MPI partitions with:

```
1 mpiexec -n 2 julia --project=. \
2   examples/distributed_periodic_poisson_bracket_maxwell.jl \
3   --partitions=2
```

The value of `-partitions` must equal the number of MPI ranks. `-partitions=N` specifies a count, while `-partition=PATH` selects an existing METIS element-partition file. In particular, `-partition=2` means “read the file named 2”; it does not request two partitions.

The generated `nx8` mesh can be run on two ranks with:

```
1 mpiexec -n 2 julia --project=. \
2   examples/distributed_periodic_poisson_bracket_maxwell.jl \
3   --partitions=2 \
4   --mesh=examples/meshes/periodic_box_2x1x1_nx8.vtk \
5   --repartition \
6   --rebuild-distributed-mesh \
7   --order=2 \
8   --esprk-order=3 \
9   --final-time=10.0 \
10  --cfl=0.5
```

The default partition path and distributed-mesh cache contain the source mesh basename, so `nx4` and `nx8` artifacts do not collide. The `-repartition` and `-rebuild-distributed-mesh` options are required when replacing a mesh while retaining the same filename.

A four-way partition is generated and run with:

```

1 mpiexec -n 4 julia --project=. \
2   examples/distributed_periodic_poisson_bracket_maxwell.jl \
3   --partitions=4 --repartition

```

9.2.2 Automatic METIS partitioning

When `-partition` is omitted, rank zero:

1. reads the legacy VTK tetrahedra;
2. calls `write_metis_mesh_from_vtk`;
3. runs the installed `mpmetis` executable for the requested number of partitions;
4. validates the zero-based element partition;
5. prepares the rank-local owned-plus-halo mesh cache.

Generated partitions are stored under `output/partitions/`. Use `-repartition` to replace an existing partition and `-rebuild-distributed-mesh` to recreate the rank-local cache. An existing partition may be selected explicitly:

```

1 mpiexec -n 2 julia --project=. \
2   examples/distributed_periodic_poisson_bracket_maxwell.jl \
3   --partitions=2 \
4   --partition=output/partitions/periodic_box_2x1x1_nx4.mesh.epart.2

```

9.2.3 Polynomial-order requirement

This driver requires DG order $N \geq 2$. For $N = 1$ on the default `nx4` mesh, all DG nodes have $x \in \{0, 0.5, 1, 1.5, 2\}$, where $\sin(2\pi x) = 0$. Nodal interpolation would therefore produce an identically zero initial field even though the continuous analytical wave is nonzero inside each interval. The driver rejects `-order=1`.

9.2.4 Diagnostics and production output

The default output directory is `output/distributed_periodic_poisson_bracket/`. The driver preserves the production I/O behavior of the PEC Poisson-bracket case:

- `energy.csv`: DG mass-matrix electric, magnetic, and total energy;
- `quadrature_diagnostics.csv`: independent-cubature field and energy errors, optical chirality, electric and magnetic charge, and linear and angular momentum;
- `fields.pvd` and parallel VTU pieces: ParaView time series;
- `integration_points_rankNNNN.csv`: final integration-point numerical and analytical values;
- `run_metadata.toml` and `partition_metadata.csv`;
- `checkpoints/stepNNNNNNNN/`: checksummed rank-local restart data.

At every cubature point, the driver evaluates the optical chirality density

$$\chi = \frac{1}{2} (\varepsilon \mathbf{E} \cdot \nabla \times \mathbf{E} + \mu \mathbf{H} \cdot \nabla \times \mathbf{H}).$$

The curls are formed with the physical element derivative operators. The diagnostics file stores the distributed integral of χ , its analytical value and error, and the numerical, analytical, and error L^2 norms of its density. The rank-local integration-point files store the corresponding pointwise densities. For the linearly polarized traveling wave used here, the analytical optical chirality is identically zero.

Restart requires the same rank count, partition, and DG order:

```
1 mpiexec -n 2 julia --project=. \
2   examples/distributed_periodic_poisson_bracket_maxwell.jl \
3   --partitions=2 \
4   --restart=output/distributed_periodic_poisson_bracket/checkpoints/step00000100 \
5   --final-time=0.5
```

The implementation has been exercised with automatic two-way and four-way METIS partitions. In the short regression runs, the centered Poisson-bracket/ESPRK discretization preserved discrete total energy to roundoff and the integrated electric and magnetic charges remained near machine precision.

9.3 Distributed nonlinear PML pulse

The executable example `examples/distributed_nonlinear_pml_maxwell.jl` demonstrates the formulation from ???. It generates $[0, 2] \times [0, 1] \times [0, 1]$ in memory, partitions its $6n_x$ tetrahedra by x slab, uses periodic boundaries in y and z , and places polynomial nonlinear PML layers at both x ends. The initial condition is a right-traveling, linearly polarized Gaussian pulse:

$$\mathbf{E} = (0, 0, g(x)), \quad \mathbf{H} = (0, -g(x), 0).$$

The default Hesthaven–Warburton upwind flux supplies a robust outer termination beyond the PML.

```
1 mpiexec -n 2 julia --project=. \
2   examples/distributed_nonlinear_pml_maxwell.jl \
3   --nx=8 --order=2 --rk-order=4 \
4   --final-time=1.3 --cfl=0.15 \
5   --pml-width=0.5 --sigma-max=12 \
6   --sigma-degree=2
```

Important options are:

-nx Number of structured x cells; there are six tetrahedra per cell. It must be at least the MPI rank count.

-pml-width

Width of each x -directed layer. It must lie in $(0, 1)$ so the two layers do not overlap.

-sigma-max and -sigma-degree

Peak damping and polynomial profile degree.

-regularization

Positive δ added to both nonlinear denominators.

-energy-every

Number of time steps between global-energy samples.

Rank zero writes `output/distributed_nonlinear_pml/energy.csv`, containing the time, step, electric energy, magnetic energy, total energy, and total-to-initial energy ratio. A representative two-rank run with the command above reduced the ratio to approximately 1.25×10^{-2} by $t = 1.3$, with no increase between sampled energy values.

9.4 Stationary PEC cube

```
1 julia --project=. examples/solver/stationary_pec_cube.jl \
2   --cells=3 --order=2 --nsteps=20 --rk=4 --cfl=0.05 \
3   --output=output/stationary_hw
```

The case uses the cube $[-1, 1]^3$, PEC walls with boundary tag 10, and a cavity mode with $\omega = \sqrt{3}\pi$. It prints energy, right-hand-side, and tangential-electric-field diagnostics and writes final VTU data.

Display its complete options with:

```
1 julia --project=. examples/solver/stationary_pec_cube.jl --help
```

9.5 Stationary central-flux ESPRK case

```
1 julia --project=. examples/solver/stationary_pec_cube_esprk.jl \
2   --cells=3 --order=2 --nsteps=20 --esprk=4 --first=H \
3   --cfl=0.05 --output=output/stationary_esprk
```

This case combines the Hesthaven–Warburton central flux with an explicit partitioned symplectic RK method. Both electric-first and magnetic-first stage orders are accepted.

9.6 Stationary Poisson-bracket ESPRK case

```
1 julia --project=. --threads=8 \
2   examples/solver/stationary_pec_cube_pb_esprk.jl \
3   --cells=3 --order=2 --nsteps=20 --esprk=4 \
4   --backend=threaded --cfl=0.05 \
5   --output=output/stationary_pb
```

The Poisson-bracket case requires `first=H`. It supports both the serial and threaded backend options.

9.7 Upwind convergence study

```
1 julia --project=. examples/solver/convergence_hw_upwind.jl \
2   --cells=2,3,4,5 \
3   --orders=1,2,4 \
4   --time=0.005 \
5   --cfl=0.1 \
6   --rk=5 \
7   --backend=serial \
8   --output=output/convergence_hw_upwind.csv
```

For a threaded run:

```
1 julia --project=. --threads=8 \
2   examples/solver/convergence_hw_upwind.jl \
3   --cells=2,3,4,5 --orders=1,2,4 --backend=threaded
```

The driver requires exactly four mesh levels. It creates deterministic jittered tetrahedral cube meshes, computes continuous error norms, estimates convergence rates, and writes CSV output.

9.8 Central-flux partitioned convergence study

```
1 julia --project=. \  
2   examples/solver/convergence_hw_central_psrk_hfirst.jl \  
3   --cells=2,3,4,5 \  
4   --orders=1,2,4 \  
5   --psrk=2 \  
6   --first=H \  
7   --output=output/convergence_hw_central_psrk_hfirst.csv
```

9.9 Poisson-bracket threaded convergence study

```
1 julia --project=. --threads=8 \  
2   examples/solver/convergence_pb_esprk_threaded.jl \  
3   --cells=3,4,5,6 \  
4   --orders=1,2,3,4 \  
5   --seconds=10 \  
6   --backend=threaded \  
7   --output=output/convergence_pb_esprk_threaded.csv
```

For this validation, the ESPRK order is fixed to spatial order plus one. A second CSV file with `_3dec` inserted before the extension is also written.

Chapter 10

Testing

10.1 Package tests

Run the non-MPI test target with:

```
1 julia --project=. -e 'using Pkg; Pkg.test()'
```

The package suite covers loading, physical weak derivatives, Maxwell volume and surface operators, formulations, backend dispatch, time integrators, energy diagnostics, and distributed-mesh consistency constructed without an MPI launcher.

10.2 MPI face permutation test

```
1 mpiexec -n 2 julia --project=. test/test_mpi_face_permutation.jl
```

This test uses two tetrahedra whose shared face has different local vertex ordering. It verifies all face-node values after MPI exchange, including quadratic edge nodes.

10.3 MPI METIS partition tests

```
1 mpiexec -n 2 julia --project=. test/test_mpi_metis_partition.jl
2 mpiexec -n 4 julia --project=. test/test_mpi_metis_partition.jl
```

These tests check owned and ghost counts, symmetric neighbor tables, deterministic communication ordering, interface orientation, and generic trace exchange.

10.4 Distributed Maxwell equivalence tests

```
1 mpiexec -n 2 julia --project=. test/test_mpi_distributed_maxwell.jl
2 mpiexec -n 4 julia --project=. test/test_mpi_distributed_maxwell.jl
```

The two-rank test compares distributed and serial results for:

- Hesthaven–Warburton central flux;
- Hesthaven–Warburton upwind flux;
- Poisson-bracket formulation;

- exchanged ghost traces;
- zero ghost residuals;
- global energy;
- one explicit RK3 step;
- one partitioned symplectic RK2 step.
- rank-local mesh persistence and reload;
- checkpoint/restart field round trips;
- run and partition metadata generation.

The four-rank chain test exercises ranks with two neighbors and verifies upwind right-hand sides, traces, ghost residuals, and RK3 stepping. The file also verifies that root input errors propagate collectively without an MPI deadlock.

10.5 Nonlinear PML tests

The package test suite checks the nonlinear source against the component equations in Appendix A of the paper, verifies the nonpositive pointwise energy contribution, exercises zero-field regularization, and confirms that a zero damping profile reproduces the ordinary Maxwell right-hand side.

The distributed test is:

```
1 mpiexec -n 2 julia --project=. test/test_mpi_nonlinear_pml.jl
```

It verifies the owned-element PML source, zero ghost residuals, and decrease of globally reduced energy after one RK4 step.

10.6 Recommended pre-commit sequence

```
1 julia --project=. -e 'using Pkg; Pkg.test()'
2 mpiexec -n 2 julia --project=. test/test_mpi_face_permutation.jl
3 mpiexec -n 2 julia --project=. test/test_mpi_metis_partition.jl
4 mpiexec -n 4 julia --project=. test/test_mpi_metis_partition.jl
5 mpiexec -n 2 julia --project=. test/test_mpi_distributed_maxwell.jl
6 mpiexec -n 4 julia --project=. test/test_mpi_distributed_maxwell.jl
7 mpiexec -n 2 julia --project=. test/test_mpi_nonlinear_pml.jl
8 mpiexec -n 2 julia --project=. examples/distributed_maxwell.jl
```

Chapter 11

Benchmarking

11.1 Single benchmark

```
1 julia --project=. --threads=8 \  
2   benchmark/solver/benchmark_backends.jl \  
3   --formulation=hw-upwind \  
4   --cells=6 \  
5   --order=3 \  
6   --seconds=3
```

Select the Poisson-bracket operator with `-formulation=pb`. The benchmark reports minimum, median, and mean times, allocations, memory, speedup, efficiency, and serial-versus-threaded right-hand-side error.

11.2 Thread scaling sweep

```
1 THREAD_COUNTS="1 2 4 8" \  
2   bash benchmark/solver/run_thread_scaling.sh \  
3   --formulation=hw-upwind --cells=6 --order=3
```

Outputs:

- `benchmark/solver/thread_scaling_raw.txt`;
- `benchmark/solver/thread_scaling.csv`.

11.3 Plot scaling data

```
1 python3 benchmark/solver/plot_thread_scaling.py
```

The script writes:

- `benchmark/solver/thread_speedup.png`;
- `benchmark/solver/thread_efficiency.png`.

Install Matplotlib separately if it is not available:

```
1 python3 -m pip install matplotlib
```

11.4 MPI performance measurements

The current repository does not contain a full MPI scaling harness. For meaningful measurements:

1. exclude Julia compilation and mesh distribution from the timed region;
2. warm up at least one right-hand-side evaluation;
3. place an `MPI.Barrier` immediately before timing;
4. reduce the maximum elapsed time across ranks;
5. report owned elements, interface faces, polynomial order, and N_{fp} ;
6. separate right-hand-side, trace exchange, energy reduction, and output costs;
7. compare strong and weak scaling.

Chapter 12

Output and Visualization

12.1 Serial VTU field output

The stationary PEC examples contain helper functions that write either:

- cell-centered fields for order zero; or
- discontinuous vertex fields for order one and above.

The discontinuous writer duplicates tetrahedron vertices per element. This is intentional: a DG solution may have two different traces at the same geometric vertex.

12.2 Partition output

`write_partition_vtk` writes the complete mesh with a partition cell array. `write_partition_piece_vtk`s writes one owned-element mesh per partition. These are mesh visualization utilities; they do not write the distributed Maxwell solution.

12.3 Distributed mesh files

The first production run may prepare one owned-plus-halo mesh shard per MPI rank:

```
1 prepare_distributed_mesh_partition(  
2     root_mesh,  
3     root_partition,  
4     "mesh_cache";  
5     comm = MPI.COMM_WORLD,  
6 )
```

Later runs bypass global mesh loading and root distribution:

```
1 distributed_dg = build_distributed_dg_from_partition(  
2     "mesh_cache",  
3     order;  
4     comm = MPI.COMM_WORLD,  
5 )
```

The directory contains `mesh_manifest.toml` and one checksummed binary piece per rank. All ranks must use the same shared filesystem and MPI rank count used when the cache was prepared.

12.4 Checkpoint and restart

```

1 write_distributed_checkpoint(
2     "checkpoints/step00000100",
3     U,
4     distributed_dg;
5     step = 100,
6     time = 0.1,
7     dt = 1.0e-3,
8 )
9
10 U, state = load_distributed_checkpoint(
11     "checkpoints/step00000100",
12     distributed_dg,
13 )

```

Each rank writes only its owned Maxwell degrees of freedom. The manifest is published atomically after every piece succeeds. Loading validates checksums, rank count, DG order, and owned global element IDs before refreshing ghost face traces. Restart currently requires the same rank count and partition.

12.5 ParaView time series

The distributed Poisson-bracket experiment writes owned elements only, preserves global element IDs and owner ranks, and represents DG discontinuities with element-local points. Use:

```

1 mpiexec -n 2 julia --project=. \
2     examples/distributed_poisson_bracket_maxwell.jl \
3     --paraview-every=10

```

Open `fields.pvd` in ParaView. It indexes `fields/fields_stepNNNNNNNNN.pvtu`, whose rank-local VTU pieces contain E , H , their magnitudes, global element IDs, owner rank, polynomial order, and physical time.

12.6 Run and partition metadata

Every production experiment writes:

- `run_metadata.toml`: configuration, Julia version, MPI size, time-step data, elapsed solve time, and final checkpoint;
- `partition_metadata.csv`: owned and ghost elements, local nodes, neighbor ranks, interface-face counts, and global element ranges;
- `paraview_series.csv`: step, time, and PVTU dataset index.

Chapter 13

Diagnostics and Troubleshooting

13.1 MPI run hangs

- Confirm every rank enters the same collective constructor and time integration sequence.
- Confirm the launch rank count matches the partition file.
- Confirm MPI.jl and `mpiexec` use compatible MPI libraries. Inspect the configured backend with `julia -project=. -e 'using MPI; MPI.versioninfo()'` and use `mpiexecjl` when the system launcher targets a different MPI implementation.
- Do not let only rank zero call `distributed_maxwell_energy`; it contains an `Allreduce`.
- Ensure all ranks use the same communication tag for a given exchange.

13.2 Wrong interface flux or rank-dependent error

- Run `test_mpi_face_permutation.jl`.
- Check tetrahedron connectivity and face orientation.
- Check that face-node generation is identical on all ranks.
- Tighten or relax `trace_tol` only after inspecting mesh scale.
- Verify that the partition includes complete face adjacency, not merely node adjacency.

13.3 Missing or incorrect boundary conditions

- Confirm the VTU file contains triangular boundary cells.
- Confirm their cell-data array is named `boundary_id`, or pass a different `boundary_tag_name`.
- Print `unique(tri_data(mesh, "boundary_id"))`.
- Register every physical PEC tag explicitly.
- Remember that tag 10 is a convention in the examples, not a hard requirement of the solver.
- For the periodic box driver, verify tags $1 \leftrightarrow 2$, $3 \leftrightarrow 4$, and $5 \leftrightarrow 6$ and matching translated triangulations.
- Periodic tags belong in the periodic exchange and should map to `MaxwellBC_None` in the physical-boundary registry.

13.4 Periodic driver fails during partitioning

- Confirm `mpmetis` is installed and visible in `PATH`.
- Confirm `-partitions` equals the `mpiexec -n` rank count.
- Use `-partitions=N` for the rank count. The singular option `-partition=PATH` is only for an existing partition file.
- If a partition contains a different number of entries than the mesh has tetrahedra, regenerate it for that mesh with `-repartition`.
- Use `-repartition` after replacing the source mesh.
- Use `-rebuild-distributed-mesh` after changing the partition.
- Use DG order two or higher for the supplied traveling wave.

13.5 Unstable time integration

- Reduce the CFL factor.
- Inspect the worst element reported by `element_size_diagnostics`.
- Check for degenerate or highly skewed tetrahedra.
- Verify positive ε and μ .
- Use the upwind Hesthaven–Warburton flux when damping unresolved jumps is acceptable.
- Confirm that the chosen RK or ESPRK order is supported.

13.6 Poisson-bracket errors

- Use `PoissonBracketFormulation()` with central flux only.
- Use the partitioned symplectic driver.
- Set `first_partition = :H`.
- Use ESPRK order one through six.

13.7 Threaded run is slower

- Increase the problem size; small meshes are overhead dominated.
- Verify the actual Julia thread count.
- Avoid oversubscribing cores in hybrid MPI/threaded runs.
- Pin ranks and threads appropriately for the machine.
- Use the benchmark harness and compare median times.

13.8 Memory use is larger than expected

- Use `build_distributed_dg_from_partition` for repeated runs, or `build_distributed_dg_from_root` during preparation.
- Remember that each rank stores owned elements plus one face halo.
- Higher polynomial order increases storage in proportion to N_p .
- Explicit RK methods store several complete right-hand sides.
- Root temporarily retains the complete global mesh and partition only while preparing rank-local mesh files.

Chapter 14

Current Limitations and Roadmap

14.1 Current limitations

- Three-dimensional tetrahedral meshes only.
- Affine straight-sided tetrahedral geometry.
- Source-free Maxwell equations.
- Material coefficients are isotropic and piecewise constant per tetrahedron; curved or nodally varying tensors are not implemented.
- The characteristic absorbing boundary is first order. It is separate from the nonlinear PML and remains useful as an outer termination.
- The nonlinear PML currently supports only the dimensionless homogeneous-vacuum system with $\varepsilon = \mu = 1$; heterogeneous material coupling and anisotropic damping tensors are not implemented.
- The nonlinear PML path supports explicit RK integration, not ESPRK, because damping breaks the conservative Poisson-bracket structure.
- No adaptive mesh refinement.
- No distributed mesh repartitioning during a run.
- Checkpoint restart requires the same MPI rank count and partition.
- Rank-local mesh files currently require a shared filesystem.
- Initial mesh-cache preparation still reads and partitions the complete mesh on root; subsequent runs are rank-local.
- Communication is nonblocking, but currently completed before local DG evaluation; volume work is not yet overlapped with trace exchange.
- Stored ghost elements currently pass through local volume evaluation before ghost residuals are cleared.

14.2 Recommended next development stages

1. Add impressed electric and magnetic sources.
2. Add anisotropic and within-element varying material tensors.

3. Extend the nonlinear PML to heterogeneous materials and validate reflection errors for oblique incidence and corner layers.
4. Add alternative higher-order absorbing boundary conditions.
5. Split the right-hand side into owned volume work and interface surface work, then overlap communication.
6. Skip ghost volume computation.
7. Add MPI strong- and weak-scaling benchmarks.
8. Add repartitioning restart and portable long-term checkpoint formats.
9. Add direct parallel ingestion from partition-aware mesh formats so cache preparation also avoids root assembly.

Appendix A

API Quick Reference

API	Purpose
<code>read_vtu_mesh(path)</code>	Read XML VTU tetrahedra, boundary triangles, and cell data into <code>RawVTUMesh</code> .
<code>DGDiscretization(mesh, order; ...)</code>	Build complete serial/threaded DG operators.
<code>SerialBackend()</code>	Select scalar local kernels.
<code>ThreadedBackend()</code>	Select Julia-threaded local Maxwell kernels.
<code>HesthavenWarburtonFormulation(kind)</code>	Strong-form Maxwell DG with central or upwind flux.
<code>PoissonBracketFormulation()</code>	Partitioned weak Maxwell operator with central flux.
<code>MaxwellBoundaryRegistry(dict)</code>	Map integer boundary tags to boundary kinds.
<code>MaxwellMaterial(epsilon, mu)</code>	Define one positive isotropic material.
<code>maxwell_element_materials(...)</code>	Map mesh material IDs to element permittivity and permeability arrays.
<code>MaxwellNonlinearPML(...)</code>	Store nodal directional damping, nonlinear parameter a , and denominator regularization.
<code>polynomial_pml_sigma(...)</code>	Evaluate a one-sided polynomial damping profile.
<code>build_maxwell_nonlinear_pml(...)</code>	Sample three damping functions at the local DG nodes.
<code>nonlinear_pml_source(...)</code>	Evaluate the six nonlinear PML source components at one point.
<code>maxwell_nonlinear_pml_rhs!(...)</code>	Assemble ordinary Maxwell terms and the serial or distributed nonlinear PML source.
<code>distributed_maxwell_nonlinear_pml_rk_step!(...)</code>	Advance one distributed explicit RK step with nonperiodic PML boundaries.
<code>distributed_periodic_maxwell_nonlinear_pml_rk_step!(...)</code>	Advance one distributed explicit RK step with periodic face pairs and nonlinear PML.
<code>interpolate_maxwell_field(...)</code>	Sample analytic electric and magnetic functions at DG nodes.
<code>estimate_maxwell_dt(...)</code>	Compute the volume-to-surface CFL estimate.
<code>maxwell_energy(...)</code>	Compute serial electromagnetic energy.
<code>run_maxwell_time_steps!(...)</code>	Advance Hesthaven–Warburton with explicit RK.
<code>run_maxwell_partitioned_symplectic_time_steps!(...)</code>	Advance a partitioned Maxwell system with ESPRK.

API	Purpose
<code>read_metis_epart(path)</code>	Read zero-based element partition IDs.
<code>build_distributed_mesh(...)</code>	Build one rank's owned-plus-halo mesh from global arrays.
<code>build_distributed_dg(...)</code>	Build distributed DG when the global mesh is replicated.
<code>build_distributed_dg_from_root(...)</code>	Distribute rank-local DG meshes from a root-owned global mesh or root-read files.
<code>build_distributed_periodic_maxwell_exchange(...)</code>	Pair periodic boundary faces and allocate periodic trace communication.
<code>distributed_maxwell_invariants(...)</code>	Reduce energy, charge, linear momentum, and angular momentum over owned elements.
<code>prepare_distributed_mesh_partition(...)</code>	Write one checksummed owned-plus-halo mesh piece per rank.
<code>build_distributed_dg_from_partition(...)</code>	Build DG by independently loading one prepared mesh piece per rank.
<code>write_distributed_checkpoint(...)</code>	Atomically write owned Maxwell fields and restart metadata.
<code>load_distributed_checkpoint(...)</code>	Validate and restore a same-partition Maxwell checkpoint.
<code>write_distributed_run_metadata(...)</code>	Write TOML run metadata and CSV partition metadata.
<code>exchange_maxwell_ghost_traces!(...)</code>	Exchange six Maxwell face traces.
<code>distributed_maxwell_energy(...)</code>	Compute owned-only local energy followed by a global sum.
<code>run_distributed_maxwell_time_steps!(...)</code>	Distributed explicit RK driver.
<code>run_distributed_maxwell_partitioned_symplectic_time_steps!(...)</code>	Distributed Poisson-bracket ESPRK driver.
<code>write_metis_mesh_from_vtk(...)</code>	Convert a supported legacy VTK tetra mesh to METIS format.
<code>write_partition_vtk(...)</code>	Write a legacy VTK mesh colored by partition.
<code>write_partition_piece_vtk(...)</code>	Write owned-element mesh pieces.

Appendix B

Command Quick Reference

```
1 # Instantiate
2 julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

3

```
4 # Package tests
5 julia --project=. -e 'using Pkg; Pkg.test()'
```

6

```
7 # Distributed Maxwell demo
8 mpiexec -n 2 julia --project=. examples/distributed_maxwell.jl
```

9

```
10 # Distributed nonlinear PML pulse
11 mpiexec -n 2 julia --project=. \
12     examples/distributed_nonlinear_pml_maxwell.jl \
13     --nx=8 --order=2 --final-time=1.3 \
14     --pml-width=0.5 --sigma-max=12
```

15

```
16 # Production Poisson-bracket run with time-series output and checkpoints
17 mpiexec -n 2 julia --project=. \
18     examples/distributed_poisson_bracket_maxwell.jl \
19     --paraview-every=10 --checkpoint-every=50
```

20

```
21 # Restart to an absolute target time
22 mpiexec -n 2 julia --project=. \
23     examples/distributed_poisson_bracket_maxwell.jl \
24     --restart=output/distributed_poisson_bracket/checkpoints/step00000050 \
25     --final-time=0.5
```

26

```
27 # MPI tests
28 mpiexec -n 2 julia --project=. test/test_mpi_face_permutation.jl
29 mpiexec -n 2 julia --project=. test/test_mpi_metis_partition.jl
30 mpiexec -n 4 julia --project=. test/test_mpi_metis_partition.jl
31 mpiexec -n 2 julia --project=. test/test_mpi_distributed_maxwell.jl
32 mpiexec -n 4 julia --project=. test/test_mpi_distributed_maxwell.jl
33 mpiexec -n 2 julia --project=. test/test_mpi_nonlinear_pml.jl
```

34

```
35 # Mesh inspection
36 julia --project=. examples/solver/read_vtu_mesh.jl mesh.vtu
```

37

```
38 # METIS preparation and partitioning
39 julia --project=. examples/write_metis.jl
40 mpmmetis examples/meshes/tet_mesh.mesh 4
```

41

```
42 # Partition visualization
43 julia --project=. examples/write_partition_vtk.jl
```

```
44
45 # Serial stationary case
46 julia --project=. examples/solver/stationary_pec_cube.jl --help
47
48 # Threaded Poisson-bracket case
49 julia --project=. --threads=8 \
50   examples/solver/stationary_pec_cube_pb_esprk.jl --help
51
52 # Backend benchmark
53 julia --project=. --threads=8 \
54   benchmark/solver/benchmark_backends.jl --help
55
56 # Thread scaling
57 THREAD_COUNTS="1 2 4 8" \
58   bash benchmark/solver/run_thread_scaling.sh
59
60 # Plot benchmark data
61 python3 benchmark/solver/plot_thread_scaling.py
62
63 # Build this guide
64 pdflatex docs/DiscoGMPI_User_Guide.tex
65 pdflatex docs/DiscoGMPI_User_Guide.tex
```

Appendix C

Reproducibility Checklist

Record the following information with every reported result:

- DiscoGMPi commit hash and working-tree status;
- Julia version and project/manifest state;
- MPI implementation and version;
- number of MPI ranks and Julia threads per rank;
- CPU/node allocation and affinity settings;
- mesh file, tetrahedron count, boundary tags, and partition file;
- polynomial order N , N_p , and N_{fp} ;
- formulation and flux;
- RK or ESPRK order and partition order;
- ε , μ , CFL, Δt , and number of steps;
- PML interface locations, $\sigma_{\max}$, profile degree, a , and denominator regularization when nonlinear PML damping is active;
- initial condition and boundary registry;
- initial/final energy and error norms;
- output and benchmark methodology.

License

DiscoGMPI is distributed under the MIT License. See `LICENSE.md` in the repository for the complete license text.